

PHASE 7 — FULL SYSTEM TESTING

System Test Report

1. Scope and Method

Phases 2-6 each tested their own layer in isolation: firmware logic reviewed line-by-line, backend hit with pytest and live curl requests, dashboard integration-tested against a live backend, RS-485 protocol cross-checked across two languages, and a multi-node load test run against the live stack. This phase instead tests the system as a whole and specifically looks for integration bugs — problems that only show up when every component is running together and something goes wrong (a process dies, a message never arrives, a client keeps retrying) — rather than re-proving each component individually.

Four kinds of testing were run, all executed for real against live processes, not simulated or projected: (1) regression re-runs of every existing automated test suite, (2) a scripted full-system scenario exercising a realistic admin session through the actual REST API a dashboard would call, (3) fault injection — deliberately killing a running component to see what the rest of the system does, and (4) a defect found by (3), fixed, and re-verified within this same phase.

2. Regression Summary

Suite	Result	Notes
Backend pytest (auth, credentials, doors, schedules, alerts, crypto, MQTT ingestion)	43 / 43 passed	39 carried over from Phase 5 + 4 new tests for the Section 4 fix
RS-485 protocol cross-language test (gcc + Python)	5 / 5 passed	Unchanged since Phase 6
Dashboard production build (npm run build)	Clean build	No changes needed

3. Full-System Scenario Test

system_tests/full_scenario_test.py drives a realistic admin session against the complete live stack simultaneously: real MQTT broker, real backend, real Building Gateway relaying real simulated door nodes over a virtual RS-485 link (same setup validated in Phase 6, now used as the substrate for this test rather than the subject of it). Every check below goes through the same REST endpoints the actual dashboard code calls.

#	Check	Result
1	Admin login succeeds	PASS
2	GET /api/doors returns 200	PASS
3-6	All 4 gateway-relayed doors (A101, A102, MAIN, SRV1) show online	PASS
7	Tamper alert published over MQTT is ingested and visible as unresolved	PASS
8	PUT /api/alerts/{id}/resolve succeeds	PASS
9	Resolved alert no longer appears in the unresolved list	PASS
10	POST /api/doors/{id}/override is accepted and reports mqtt_delivered=true	PASS
11	Override round-trips through MQTT -> gateway -> simulated node -> event -> DB and appears in /logs	PASS
12	6th consecutive bad login against a live account is rejected with HTTP 429	PASS

All 12 checks passed on the first full run after the fix described in Section 4 was applied (see Section 5 for the one defect this phase found before that fix).

4. Defect Found: Door Status Never Recovers From a Gateway Crash

4.1 How it was found

Fault injection: with the full stack running and all 4 doors correctly reporting online through the Building Gateway, the gateway process was killed outright (kill -9, simulating a crash or power loss rather than a clean shutdown) and the system was watched to see what happened to those doors' status.

4.2 What went wrong

Before this phase, a door's online flag only ever went back to false because of an explicit "offline" message — either a directly-connected node's MQTT last-will firing on ungraceful disconnect, or the gateway itself publishing "offline" after missing several polls (both of these were tested and working as of Phase 6). Killing the gateway process bypasses both paths: there is no one left to publish anything for the RS-485 nodes it was relaying. Those 4 doors stayed online=true in the database indefinitely — a dashboard would keep showing all doors healthy even though the entire RS-485 backbone had gone dark. For an access-control system, a confidently wrong "online" status is worse than an honest "unknown."

4.3 Fix

app/services/staleness_watchdog.py: a periodic, transport-agnostic sweep that flips any door still marked online to offline if its last_seen timestamp is older than DOOR_STALE_AFTER_SECONDS (default 30s, checked every DOOR_STALENESS_CHECK_INTERVAL_SECONDS = 10s by default). It doesn't matter whether a door talks directly or through a gateway — it only looks at when it was last actually heard from, so it closes the gap for any future transport, not just this one. Started/stopped alongside the existing MQTT service in main.py's lifespan.

4.4 Verification

Check	Result
Unit tests (4 new, tests/test_staleness_watchdog.py): stale door flagged, recent door left alone, never-reported door (last_seen NULL) left alone, already-offline door not re-reported	Fixed & verified
Live re-run of the fault injection: gateway killed at T+0, backend log showed "Marked 4 door(s) offline" by T+4s, confirmed via GET /api/doors showing online=false for all 4	Fixed & verified

Configured with a 3-second staleness threshold and a 1-second check interval for this specific test run (to keep the test fast); production defaults are 30s / 10s, tunable via .env without a code change.

5. Requirements Traceability

Requirement (from the proposal / System Design Document)	Test evidence
Credential-based access control (card, optional 2nd-factor biometric)	Firmware CARD_SCAN/BIOMETRIC_CHECK state logic (reviewed); backend credential lookup + event logging (pytest, live scenario test)
Cryptographic (not UID-only) card authentication	DESFire AES mutual-auth handshake (Phase 5, reviewed against spec, unverified on real hardware)
Tamper detection with an active response, not just an alert	Firmware tamper lockout (Phase 5, reviewed); ingestion + resolution round-trip (Section 3, checks 7-9, PASS)
Fail-safe / fail-secure lock behavior + REX + mechanical override	Firmware state machine review (Phases 2, 5) — life-safety paths independent of firmware/network state
Credential data encrypted at rest	app/crypto.py, Fernet + blind index (Phase 5 pytest, still 43/43 passing)

Requirement (from the proposal / System Design Document)	Test evidence
Role-based access control enforced server-side	require_admin dependency (pytest); override/resolve endpoints require admin (Section 3)
Login brute-force protection	app/rate_limit.py (pytest); live 429 check (Section 3, check 12)
Hybrid RS-485 backbone + Wi-Fi fallback	Protocol cross-language test (Section 2); gateway relay proven live (Section 3, checks 3-6, 10-11)
Multi-node scalability	Phase 6 concurrent load test (288/288 events, 0 errors, 479 req/s REST) — not re-run this phase, no relevant changes since
Live dashboard status accuracy	Section 4 — the one gap found here, now fixed and verified

6. Known Limitations Carried Forward

Unchanged from prior phases, listed together here for a single point of reference rather than repeated across documents:

- The ESP32 firmware — DESFire AES handshake (Phase 5) and RS-485 primary/fallback logic (Phase 6) — has never been compiled or run on real hardware in this environment. Both are reviewed line-by-line against their respective specs/APIs; the RS-485 wire format itself (not the firmware's use of it) is independently verified via the gcc/Python cross-language test.
- No real RS-485 electrical bus (multi-drop wiring, termination, real transceivers, real cable runs) has been tested — only a virtual serial link standing in for it.
- All cards at a site share one AES key (no per-card key diversification) — flagged since Phase 5 as the top security item for a production deployment.
- SQLite (the project's zero-setup default) was load-tested at pilot scale (16 nodes) in Phase 6 with zero errors, but is not expected to hold up at full building scale — PostgreSQL remains the recommended production database, per the original proposal.
- JWT session revocation is not implemented; a leaked token remains valid until its 60-minute expiry.

7. Conclusion

Every previously-built layer of the system still works together after Phase 6's changes (43/43 backend tests, 5/5 protocol tests, clean dashboard build). A full-system scenario test covering login, live door status, tamper handling, remote override, and rate limiting passed all 12 checks end to end through the real REST API. Fault injection — deliberately killing the Building Gateway mid-run — found one real defect (stale door status surviving a component crash), which was fixed and re-verified live within this same phase rather than merely logged as a known issue. Combined with the limitations explicitly carried forward in Section 6, this gives an accurate, evidence-based picture of what the system does and doesn't guarantee going into Phase 8 (documentation) and Phase 9 (presentation/defense prep).